

Estimating Attack Complexity

2025

1 Introduction

Recall that the total power consumption of a device depends on the power consumption of all the logic cells and their interconnects making up the circuit. Hence, the total power consumption essentially depends on the number of logic cells in a circuit, the connections between them, and the technology based upon the cells are built.

For each single power measurement, we refer to the operation-dependent component of the point as P_{op} , and to the data-dependent component as P_{data} . Besides P_{op} and P_{data} , each point of a power trace also depends on what is going on the other, possibly data and operation independent “stuff”, which we shall simply call “noise” P_{noise} . The components P_{op} , P_{data} , and P_{noise} are additive. Therefore, it is possible to simply model each point of a power trace as the sum of these components:

$$P_{total} = P_{op} + P_{data} + P_{noise}. \quad (1)$$

In the rest of this document we will develop an understanding of how the relationship between signal and noise impacts on attack outcomes/attack complexity. With this, we can analyse how certain types of countermeasures impact on attack outcomes/attack complexity. There are multiple factors that impact on the overall attack complexity: the number of traces for model building (if we are allowed to do so), the number of traces for an attack with some given success criterion, the size of the key guess, etc. The factor that is ultimately used to guide decisions about what countermeasures to deploy and how, is the number of traces during the attack. Consequently, we will focus on how signal and noise impact on the number of traces that are required for a “successful” attack.

2 Analysing Attack outcomes

At this point, we have developed a framework for understanding and also practically configuring differential style attacks. Given that a number of choices are available for the different steps, a number of obvious questions arise: how do we judge the quality of attacks; what is the impact of different choices on the quality; can we predict, based on trace characteristics, how well attacks should work; and is there a “best” attack?

Answers to these questions are in fact a matter of ongoing research, and I aim here to give you a fair summary of what we (= the community of researchers and practitioners) know for certain. Thereby the emphasis is to understand how countermeasures impact on attack outcomes.

2.1 Signal to Noise ratio

We introduced earlier to concept of data and operation dependency via making them explicit as variables in the total power consumption. Besides these components we also explicitly named a third component: noise (i.e. variation in power that is due to other things going on in the circuit). In differential style attacks we are targeting the same operation (i.e. this variable is constant) across the different traces. The variation in these traces is due to data (aka “the signal”) and noise. Intuitively we would expect that the ratio between signal and noise will have a strong impact on how well attacks work in practice.

But how should we quantify signal (and noise)? This is typically done by the so-called **electric power**¹ of a signal. The power $p(t)$ of an (electrical) signal (this includes the part of the signal that we call the noise) is defined as the product of current $i(t)$ and voltage $v(t)$, i.e. $p(t) = i(t) \cdot v(t)$. Ohm’s law relates voltage to current and resistance R , so $v(t) = R \cdot i(t)$. Therefore the power can be expressed as $p(t) = i(t)v(t) = R \cdot i^2(t)$.

Depending on the signal, which in turn depends on the data that the circuit represents at time t , the power of the signal will be stronger or weaker. Therefore, it is often useful to consider the expected value of the power as a measure. The expected value of the power is given as the power at time t observed for many measurements $E[p(t)] = R \cdot E[i^2(t)]$ (for some specific input to the circuit).

When working with power traces (these are essentially measurements $i(t)$, then we can compute the mean for each trace point, and subtract it from all traces: this is called “mean centering”, and it just ensures that each trace that we work with is located around the x-axis. Because mean centering is only a linear transformation on the traces, it does not loose any information. It is useful though because if the mean of a random variable is zero, then it’s the mean of its square is the variance. This fact implies that after mean-centering traces (i.e. $E[i(t)] = 0$) then $E[i(t)^2] = Var(i(t))$.

¹Notice the unfortunate clash of terminology here: power traces are in fact typically measurements of instantaneous voltage, which relates to the current consumption.

In (statistical) signal processing, the **signal-to-noise ratio (or SNR)** is defined as the ratio of the average power of a signal (meaningful information) to the average power of background noise at some point t :

$$SNR(t) = \frac{Var(Signal(t))}{Var(Noise(t))} \quad (2)$$

In case of a differential style attack, the *signal* corresponds to what the adversary predicts, and therefore exploits. This may sometimes only be a fraction of P_{data} . Therefore we make this explicit and use the notation P_{exp} to refer to the power consumption that is due to the prediction. The noise component that we identified before, which is independent of the signal, retains its meaning. It is important to keep in mind that there may hence be a fraction of the data dependent power consumption which is not captured by an adversary and this makes calculating SNRs potentially tricky in practice. The specific SNR then for a differential attacker is:

$$SNR(t) = \frac{Var(P_{exp}(t))}{Var(P_{noise}(t))} \quad (3)$$

It is possible, if the leakage distribution of P_{exp} is known (i.e. one has done a statistical analysis), to calculate the SNR for trace points. This calculation is straightforward if $P_{data} = P_{exp}$, and I just show you some results in Figure 1. The first plot of Figure 1 shows 9 average traces of the simple microcontroller that forms the basis for all experiments in these notes. These 9 traces were obtained by sampling the power consumption of a memory transfer with uniformly distributed data. From the randomly sampled data, I then collected all traces associated with 0, 1, 2, ... Hamming weight, averaged over each class, and then presented the resulting average traces. You can see that the traces differ in several regions of the figure, in particular when the signal rises or falls sharply (i.e. near a clock edge). The data dependency, i.e. the signal, is exactly in these moments where these differences occur.

The second plot shows the standard deviation of the power traces for the case that the microcontroller processes some fixed data, i.e. this plot shows the standard deviation of the electronic noise P_{noise} . Obviously the noise isn't the same at the different moments in time, and it shows a pattern that coincides with the clock edges (the faster signals change in a circuit the more power is consumed and the more variation there is).

The third plot shows the SNR. The SNR is highest where we expect it to be: in the points where the average traces differ most, the SNR is highest.

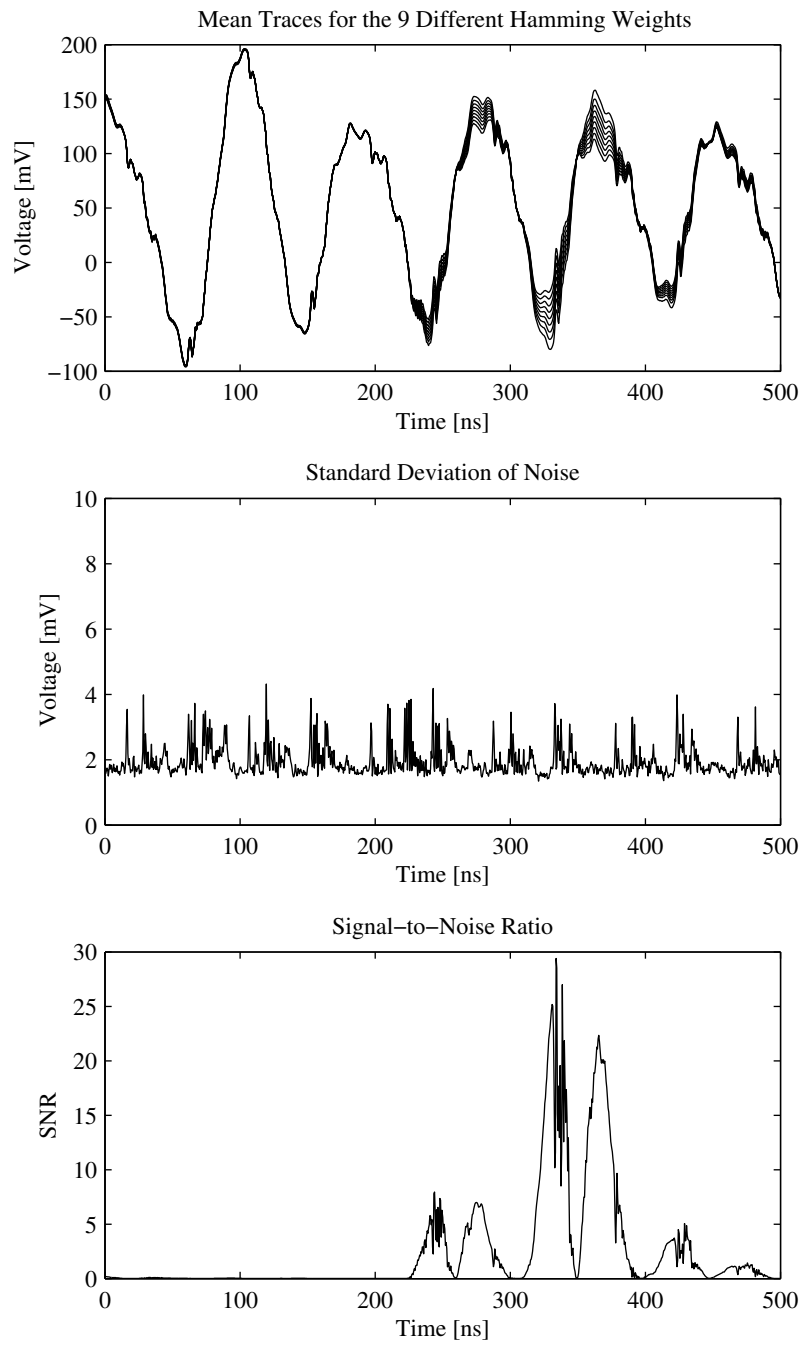


Figure 1: The signal levels, the standard deviation of the noise, and the SNR when attacking a uniformly distributed 8-bit data value on our microcontroller.

The SNR definition as we are going to use it in this handout will turn out to be useful to predict and understand DPA outcomes. However it does require to estimate the distribution of P_{exp} , which is not always possible (e.g. modern mid-range microprocessors are considerably more complex than the device that was used to generate the pictures here, often in modern devices P_{exp} is a function of not just the current but also previous data values). In such cases an alternative definition for the SNR has been used in the community which uses the reciprocal of the coefficient of variation, i.e., the ratio of mean to standard deviation of a signal: $SNR = \frac{\mu}{\sigma}$, where μ is the mean of P_{data} and σ is an estimate of the standard deviation of the noise. Another “fudge solution” is to ignore the actual distribution of P_{exp} and just group traces according to simple characteristics of the input, e.g. HW of a nibble.

2.2 Distinguisher statistic

The main attack strategy that we consider is called a “standard DPA attack”.

DPA is motivated by the intuition that the model predictions under the correct key hypothesis should “be closer” to the true trace measurements than the model predictions under an incorrect key hypothesis. We use a statistical tool called a distinguisher (which is a comparison statistic) to quantify this relationship.

A distinguisher **Dist** is a function that is applied to the measurements and the hypothesis-dependent predictions.

An intuitive choice for the comparison statistic *Dist* is the (Pearson) correlation. It is based on the covariance (which is used to express the linear relationship between two statistical variables). The definition of the covariance is given in (4) and an equivalent form is given in (5). The latter definition (5) shows that the covariance is also related to the concept of statistical dependence. If X and Y are statistically independent, then $E(XY) = E(X) \cdot E(Y)$, and therefore $Cov(X, Y) = 0$. For normally distributed X and Y also the converse holds. This means, if $Cov(X, Y) = 0$, then X and Y are independent.

$$Cov(X, Y) = E[(X - E[X]) \cdot (Y - E[Y])] \quad (4)$$

$$= E[XY] - E[X] \cdot E[Y] \quad (5)$$

The Pearson correlation coefficient ρ is essentially a “normalised” version of the covariance (i.e. it can only take values between plus and minus one: $-1 \leq \rho \leq 1$). Notice that the definitions of the covariance, correlation, etc. are all based on knowing the distribution of X and Y . This means that if and only if we have the distribution of X and Y then these quantities can be calculated. If we do not have the distributions of X and Y , but we can observe outcomes from sampling from X and Y , we cannot calculate any of these quantities. But we can estimate them using well established estimators. For the Pearson correlation coefficient ρ the corresponding estimator is called r , and it is given below.

$$\rho(X, Y) = \frac{\text{Cov}(X, Y)}{\sqrt{\text{Var}(X) \cdot \text{Var}(Y)}} \quad (6)$$

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x}) \cdot (y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \cdot \sum_{i=1}^n (y_i - \bar{y})^2}} \quad (7)$$

2.3 Number of Needed Traces

In DPA attacks, the correlation coefficients between the columns of the predictions $\mathbf{H}$ and traces $\mathbf{T}$ are estimated based on the D elements of these columns. The more traces an attacker acquires, the more precisely the estimated correlation coefficients $r_{i,j}$ of the matrix $\mathbf{R}$ match the actual correlations $\rho_{i,j}$.

Often the pertinent question in the context of DPA attacks is how many traces does an attacker need to acquire in order to determine the key that is used by the device? In case of the correlation coefficient this means: how many traces are needed in order to find out which column of $\mathbf{H}$ has the strongest correlation to a column of $\mathbf{T}$? Although this question looks pretty simple at first sight, giving a precise answer is not straightforward without knowledge of the device's leakage characteristics. In fact, we can only provide a rule of thumb in this case.

Recall that (assuming a good enough device leakage model is used), a successful DPA means that there is a significant peak for the point in time when the key dependent intermediate is processed (aka row ck and column ct of $\mathbf{R}$). It may happen that at time ct more key hypotheses than just ck show some peaks — but we now make the deliberate, simplifying assumption that the peaks associated with other key hypothesis are insignificant: we “pretend” they are 0. Therefore, a DPA attack will be successful if the correlation associated with ck can be distinguished from 0.

2.3.1 Estimate for the number of traces

The number of traces n that is necessary to assert with a confidence of $1 - \alpha$ that the distribution associated with the estimation of ρ_0 is different from the estimation of ρ_1 is:

$$n = 3 + 8 \cdot \frac{z_{1-\alpha}^2}{\left(\ln \frac{1+\rho_0}{1-\rho_0} - \ln \frac{1+\rho_1}{1-\rho_1} \right)^2} \quad (8)$$

If we may assume that the correlation for incorrect key candidates is close to zero, i.e. $\rho_0 = 0$, then we can simplify this further.

Table 1: The calculated number of traces n for different values of $\rho_{ck,ct}$.

$\rho_{ck,ct}$	$n_{\alpha=0.0001}$	$\rho_{ck,ct}$	$n_{\alpha=0.0001}$	$\rho_{ck,ct}$	$n_{\alpha=0.0001}$
0.900	16	0.090	3 400	0.009	341 493
0.800	26	0.080	4 307	0.008	432 206
0.700	40	0.070	5 630	0.007	564 519
0.600	61	0.060	7 668	0.006	768 378
0.500	95	0.050	11 049	0.005	1 106 471
0.400	157	0.040	17 273	0.004	1 728 870
0.300	292	0.030	30 720	0.003	3 073 559
0.200	676	0.020	69 140	0.002	6 915 526
0.100	2 751	0.010	276 606	0.001	27 662 152

Now we can provide a simplified “rule of thumb” for the number of needed power traces. The number of traces n that are needed to mount a successful DPA attack can be approximated as follows:

$$n = 3 + 8 \frac{z_{1-\alpha}^2}{\ln^2 \frac{1+\rho_{ck,ct}}{1-\rho_{ck,ct}}} \quad (9)$$

This formula may not be very helpful at first because you may argue that nobody knows $\rho_{ck,ct}$ a priori. However, there are a number of estimation techniques that practitioners will use to determine this value. Furthermore, one can view this value as a kind of “security” parameter: designers/implementers will do their best to make $\rho_{ck,ct}$ as small as possible. The “security level” device is then the “remaining” correlation, which can be directly mapped to a number of needed traces for DPA style attacks, like shown in Tab. 1. Table 1 shows how the number of traces increases when the correlation decreases.

The number of traces when working with correlation estimates also relates nicely to the SNR. To show this we use our notation from the “five steps” description from DPA style attacks (H_i are the predictions, and P_{total} are the measured traces), and the fact that the component P_{noise} has zero mean):

$$\begin{aligned}
\rho(H_i, P_{total}) &= \\
&= \rho(H_i, P_{exp} + P_{noise}) \\
&= \frac{E[H_i \cdot (P_{exp} + P_{noise})] - E[H_i] \cdot E[P_{exp} + P_{noise}]}{\sqrt{\text{Var}(H_i) \cdot (\text{Var}(P_{exp}) + \text{Var}(P_{noise}))}} \\
&= \frac{E[H_i \cdot P_{exp} + H_i \cdot P_{noise}] - E[H_i] \cdot (E[P_{exp}] + E[P_{noise}])}{\sqrt{\text{Var}(H_i) \cdot \text{Var}(P_{exp})} \sqrt{1 + \frac{\text{Var}(P_{noise})}{\text{Var}(P_{exp})}}} \\
&= \frac{E[H_i \cdot P_{exp}] - E[H_i] \cdot E[P_{exp}]}{\sqrt{\text{Var}(H_i) \cdot \text{Var}(P_{exp})} \sqrt{1 + \frac{\text{Var}(P_{noise})}{\text{Var}(P_{exp})}}} \\
&= \frac{\rho(H_i, P_{exp})}{\sqrt{1 + \frac{1}{SNR}}} \tag{10}
\end{aligned}$$

We can simplify the relationship further by assuming that the adversary has done a really good job when modelling: say that their predictions are a perfect fit to the signal, i.e. $\rho(H_i, P_{exp}) = 1$. Then, we find that

$$\frac{1}{\rho^2} \approx \frac{1}{SNR}.$$

Now we can bring this observation together with our number of traces estimate from before, and we find that:

$$n \approx \frac{28}{\rho_{ck,ct}^2} \approx \frac{1}{SNR} = \frac{\text{Var}(P_{noise})}{\text{Var}(P_{exp})}. \tag{11}$$

- Halving $\rho_{ck,ct}$ (aka SNR) means that four times more traces are needed.
- Reducing the signal/increasing the noise equally impacts on n .

3 Impact of countermeasures

We developed these statistical relationships for one main reason: to assess the impact of countermeasures on the cost of a differential input attack. Although the “cost” in practice does not only depend on the number of traces that are required to succeed, the number of traces is one of the main factors that are considered by practitioners. As explained before a precise estimate does necessitate profiling, and is extremely dependent

on the quality of measurements (so often this number does not easily translate from one setup to another), we can use our “rule of thumb” to assess the “relative” impact of countermeasures.

3.1 Hiding

Hiding countermeasures that are not based on changing the leakage characteristics of a device, are often realised by randomly inserting instructions, or reordering instructions, so that the “target” of a differential attack occurs at different points in time in consecutive executions of the algorithm.

Recall that the correlation between the hypothetical power consumption for the correct key hypothesis H_{ck} and the power consumption at the moment of time ct is given by $\rho_{ck,ct}$ which corresponds to $\rho(H_{ck}, P_{total})$. To account (in our notation) explicitly for the fact that for some traces at time point ct something “random” is happening, we add a “hat”: $\hat{P}_{total}$. We use $\hat{p}$ to quantify the probability that the “right” thing happens in time point ct , and therefore $1 - \hat{p}$ is the probability that something “random” is happening. Either the right thing happens or something random, therefore we can write the covariance between H_{ck} and $\hat{P}_{total}$ as follows:

$$Cov(H_{ck}, \hat{P}_{total}) = \hat{p} \cdot Cov(H_{ck}, P_{total}) + (1 - \hat{p}) \cdot Cov(H_{ck}, P_{other})$$

We can plug this covariance into the definition for $\rho(H_{ck}, P_{total})$ as shown in (12) and simplify the expression because $Cov(H_{ck}, P_{other}) = 0$, and P_{other} and P_{total} are independent.

$$\begin{aligned} \rho(H_{ck}, \hat{P}_{total}) &= \frac{\hat{p} \cdot Cov(H_{ck}, P_{total}) + (1 - \hat{p}) \cdot Cov(H_{ck}, P_{other})}{\sqrt{Var(H_{ck}) \cdot Var(\hat{P}_{total})}} \\ &= \frac{\hat{p} \cdot Cov(H_{ck}, P_{total})}{\sqrt{Var(H_{ck}) \cdot Var(\hat{P}_{total})}} \cdot \sqrt{\frac{Var(P_{total})}{Var(P_{total})}} \\ &= \rho(H_{ck}, P_{total}) \cdot \hat{p} \cdot \sqrt{\frac{Var(P_{total})}{Var(\hat{P}_{total})}} \end{aligned} \tag{12}$$

One way of dealing with traces in which the target intermediate value does not occur at some fixed time across all traces is to use alignment techniques. Trace alignment is about ensuring that the targeted intermediate value occurs at some small interval across all traces. Discussing alignment fully is beyond the scope of this unit, but a basic technique is to “integrate” trace points over some set interval. The exact type of “integration” can vary, and for simplicity we will assume that an adversary will simply

sum over l trace points. In principle the target intermediate value can occur in any of the l points. Therefore we say that it will occur in the first P_1 (we could have chosen any of the l points), and we analyse the correlation in that respect. All other points are considered to be noise, i.e. they are statistically independent of each other and also of H_{ck} . Therefore, it holds for $i = 2, \dots, l$ that $E(H_{ck} \cdot P_i) - E(H_{ck}) \cdot E(P_i) = 0$. The correlation coefficient for the correct key hypothesis is thus:

$$\begin{aligned}
\rho(H_{ck}, \sum_{i=1}^l P_i) &= \frac{E(H_{ck} \cdot \sum_{i=1}^l P_i) - E(H_{ck}) \cdot E(\sum_{i=1}^l P_i)}{\sqrt{\text{Var}(H_{ck}) \cdot (\text{Var}(\sum_{i=1}^l P_i))}} \\
&= \frac{E(H_{ck} \cdot P_1 + H_{ck} \cdot \sum_{i=2}^l P_i) - E(H_{ck}) \cdot (E(P_1) + E(\sum_{i=2}^l P_i))}{\sqrt{\text{Var}(H_{ck}) \cdot \text{Var}(P_1)} \sqrt{\frac{\sum_{i=1}^l \text{Var}(P_i)}{\text{Var}(P_1)}}} \\
&= \frac{E(H_{ck} \cdot P_1) - E(H_{ck}) \cdot E(P_1)}{\sqrt{\text{Var}(H_{ck}) \cdot \text{Var}(P_1)} \sqrt{\frac{\sum_{i=1}^l \text{Var}(P_i)}{\text{Var}(P_1)}}} \\
&= \frac{\rho(H_{ck}, P_1)}{\sqrt{\frac{\sum_{i=1}^l \text{Var}(P_i)}{\text{Var}(P_1)}}} \tag{13}
\end{aligned}$$

If the variances in the trace points are roughly equal, then we can simplify this expression further and it becomes

$$\rho(H_{ck}, \sum_{i=1}^l P_i) = \frac{\rho(H_{ck}, P_1)}{\sqrt{l}}. \tag{14}$$

This last equation tells us that the impact of misalignment on the correlation is that it reduces it by a factor that corresponds to the square root of the misalignment. Thus the way that this countermeasure scales the correlation of the correct key (which is our main factor in the “rule of thumb” for the number of needed traces) is not even linear.

Example. Suppose that we create some temporal misalignment of the SubBytes lookups by performing some of them in a random order. The top two plots in Fig. 2 show the situation of an attack on the first SubBytes if the misalignment is turned off. The second row show the outcome of an attack if we just randomly swap the first SubBytes operations. The third row shows the outcome of an attack if we randomly swap around four bytes, and the last row shows the result of randomly swapping around 8 bytes.

The left column shows attack outcomes of an naive attack: i.e. the adversary does not integrate over trace points. As expected the correlation drops linearly with the displacement. The right column shows attack outcomes of an adversary who integrates

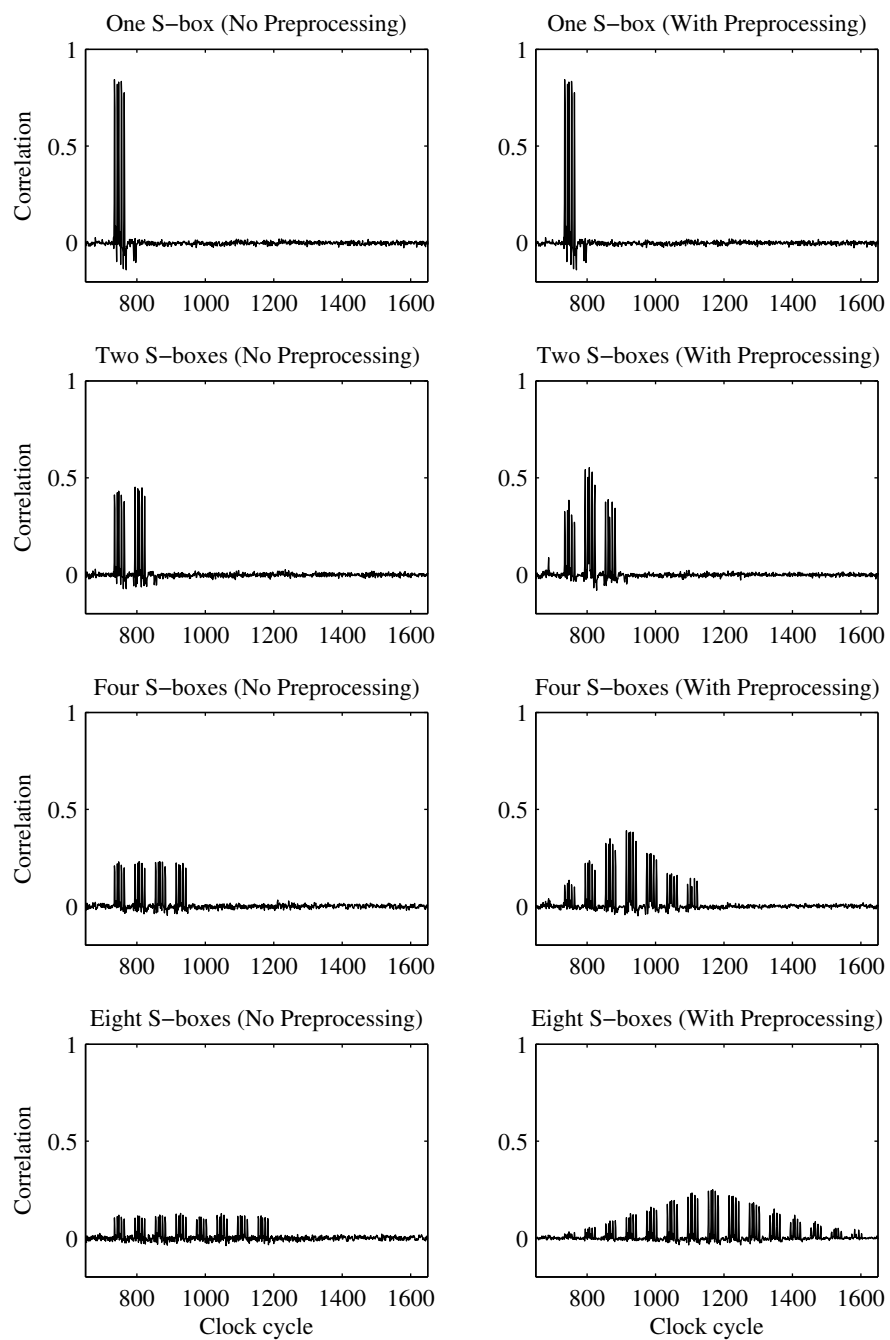


Figure 2: Misalignment and trace point integration

over several trace points. As expected the correlation drops with the square root of the displacement.

3.2 Masking

Masking is based on secret sharing sensitive variables, and computing the entire round function on the shares only. The security argument for masking consists of two parts:

1. **No leakage below threshold:** Masking schemes typically argue why no reconstruction of the secret is possible below a given threshold. We will look at this part of the security argument shortly.
2. **Exponential cost increase at threshold:** Obviously whenever the leakage of sufficiently many shares is recombined, an attack becomes possible. There is the generic claim that recombination of d shares increases the effort by an exponential factor of d .

We now consider the second part of the security argument: that the cost of an attack increases exponentially. We do this using the statistics that we have introduced already to understand cost, when understood as the number of traces needed for a successful attack.

Let us assume that we are in the situation where the “rule of thumb” applies: i.e. we have sufficiently much noise in relation to the signal. Then we know that the correlation of the correct key is proportional to the inverse of the SNR:

$$\rho = \frac{\text{Var}(P_{\text{noise}})}{\text{Var}(P_{\text{signal}})}.$$

Let us further assume that the signal and noise characteristics of individual trace points does not change when we implement a round based on shares. This makes sense because we will still be using the same instructions, thus the power consumption characteristics remain the same.

The reason why we cannot succeed with a differential input attack on single trace points is because individual trace points depend (in theory) on single shares. Because shares are random values, there is no data dependency to exploit. Only when we combine sufficiently many trace points, and thereby sufficiently many shares, we can hope to succeed in an attack.

What does it mean to “combine” trace points? There are many different ways of “combining” trace points: one can add, subtract, square and add/subtract, etc. It turns out

that when a device leaks proportional to the Hamming weight, then a really good way of combining is by doing what is called a “mean-free product” between trace points. This works as follows for two trace points x and y :

$$z = (x - E(x))(y - E(y)).$$

The attack then works on a new trace that is based on points z , and it proceeds like the attacks that we studied before.

We now look at the effect that this trace transformation has on estimated the number of needed traces as per our rule of thumb. The number of traces is determined by the variance of the signal and the variance of the noise after mean-free product combining. The points z are the product of two random variables. Because an adversary does not know which two trace points to combine, they have to apply the mean-free product combination exhaustively to (a window of points of) a trace. The resulting trace(s) consist of many points z some of which will be highly statistically dependent (from one another and from the actual intermediate values) and some of which will be independent. Reasoning about products of dependent variables is tricky. As we are working with “rules of thumb” anyway, we will now assume that x and y are independent of each other, in order to reason about their product.

The best z has some signal (the product of the underlying distributions) that is given as $E(x)E(y)$, but the adversary still only exploits part of it, thus typically the variable P_{data} has not changed. The variance of the best z is given as

$$Var(xy) = (\sigma_x^2 + \mu_x^2)(\sigma_y^2 + \mu_y^2) - \mu_x^2\mu_y^2 \geq \sigma_x^2\sigma_y^2.$$

Thus the variance of z is at least as high as the $\sigma_x^2\sigma_y^2$, if $\sigma_x^2 \sim \sigma_y^2$. then we have that the SNR of z is $\frac{Var(P_{exp})}{Var(z)} = \left(\frac{Var(P_{exp})}{Var(P_{noise})}\right)^2$.

If $Var(P_{noise})$ is the dominating factor (which is our assumption) then this implies an exponential increase by a factor of two. The variance of multiple independent points after mean-free product combining can be derived via simply extending the above formula to multiple variables.

Combining d trace points leads to an exponential increase in the number of needed traces:

$$n = \left(\frac{Var(P_{noise})}{Var(P_{exp})}\right)^d.$$

Thus in comparison to hiding, masking offers (in theory), a much steeper increase in the cost of an adversary (sublinear vs. exponential), which is why most real world products implement masking (often with some cheap hiding alongside to change the SNR favourably so masking has an even greater effect).